Instructions

1. Use MATLAB or Python. Implement the planning algorithms yourself. If possible try not to use any built-in planners and shortest-path functions (for example plannerAStar, plannerRRT, graph/shortestpath, or the path functions of networkx). Standard functions for arrays, plotting and solving differential equations may be used.
2. Use the data given in each task. If a parameter is not given, assume a suitable value and state it.
3. Submit one PDF report (not handwritten) named RollNo_nameinitial.pdf. For each task it must contain: the code (readable screenshots or text), the output plots with title, axis labels, units and legend, the results in tables, and a very short discussion of the observations.
4. Every student must write and run their own code. Copied reports will not be evaluated.

Task 1 Grid-based path planning: Dijkstra and A*

A point mobile robot moves on a 30 × 30 occupancy grid. The cells are indexed by (x, y), with x, y = 0, 1, …, 29. The obstacle cells are the rectangular blocks given below (limits are inclusive). The start cell is (2, 2) and the goal cell is (27, 27).

Obstacle	x range	y range
R1	8 to 10	0 to 18
R2	15 to 17	10 to 29
R3	22 to 24	0 to 20

Motion model: 8-connected moves, with a cost of 1 for a horizontal or vertical move and $\sqrt{2}$ for a diagonal move. A diagonal move is allowed only if both of the orthogonal neighbouring cells are free (no corner cutting). A* uses the Euclidean distance to the goal as the heuristic.

(a) Implement Dijkstra's algorithm and the A* algorithm for the above problem.

(b) For each algorithm, plot the grid showing the obstacles, the start, the goal, the cells expanded by the algorithm and the final path.

(c) Compare the two algorithms in a table: path cost, number of nodes expanded, computation time (mean of 20 runs), number of turns and total heading change of the path (rad), which is a measure of path smoothness.

(d) Smooth the A* path using line-of-sight shortcutting. Plot the smoothed path and report its length, number of turns and total heading change.

(e) Discuss why A* expands fewer nodes than Dijkstra's algorithm, and which path is smoother and why.

Task 2 Joint-space trajectory of a 2-link planar manipulator

A 2-link planar manipulator has link lengths $l_1 = 1.0$ m and $l_2 = 0.8$ m. The angle θ_1 is measured from the x-axis and θ_2 is measured relative to link 1. The manipulator moves from the start configuration $(\theta_1, \theta_2) = (30°, 45°)$ to the end configuration $(\theta_1, \theta_2) = (60°, 90°)$, starting and stopping at rest. Use a cubic polynomial in joint space for each joint, with a total time T = 3 s for both joints and a sampling time of 0.01 s.

(a) Compute the cubic polynomial coefficients of each joint and generate the joint position, velocity and acceleration.

(b) Plot the joint angles, joint velocities and joint accelerations against time (in deg, deg/s and deg/s²) and mark the peak values.

(c) Using forward kinematics, plot the end-effector coordinates x and y against time and the Cartesian path of the end-effector in the x–y plane, with the start and end points marked. Draw the arm configuration at t = 0, T/4, T/2, 3T/4 and T on the same plot.

Task 3 Sampling-based planning: RRT or PRM

A point robot moves in a 100 × 100 workspace containing the circular obstacles given below. The start is (5, 5) and the goal is (95, 95). Implement **either** RRT **or** PRM.

Obstacle	Centre x	Centre y	Radius
O1	30	30	10
O2	60	70	12
O3	70	30	8
O4	20	75	9
O5	50	50	7

- RRT: step size 5 units, goal bias 5 %, goal tolerance 5 units, maximum 5000 iterations.
- PRM: 300 random collision-free samples, each connected to its neighbours within a radius of 15 units (or to its 10 nearest neighbours). Find the shortest path on the roadmap using your own implementation of Dijkstra's algorithm or A*.

A new node is accepted (RRT) or an edge is added (PRM) only if the straight segment joining the two points is free of collision with all the obstacles.

(a) Write a collision-checking function for a line segment against a circular obstacle.

(b) Implement the chosen algorithm.

(c) Plot (i) the random samples and the expanding tree (or the roadmap), and (ii) the final collision-free path overlaid on the obstacles, start and goal.

Task 4 Dynamics of a 2-link planar manipulator

Consider the manipulator of Task 2 (l_1 = 1.0 m, l_2 = 0.8 m, same angle conventions) moving in the vertical plane. Both links are uniform slender rods with the centre of mass at the mid-point of the link and the moment of inertia about the centre of mass $I = ml^2/12$. The masses are m_1 = 2.0 kg and m_2 = 1.5 kg. Gravity (g = 9.81 m/s²) acts in the −y direction. Neglect friction.

(a) Forward dynamics: with $\tau = 0$, release the arm from rest at $q = (0°, 0°)$ (arm horizontal and fully stretched). Simulate for 5 s using an ODE solver (ode45). Plot θ_1 and θ_2 against time and the path of the end-effector. Plot the total energy (kinetic + potential) against time to verify energy conservation.

(c) Inverse dynamics: compute the joint torques required for the manipulator to follow the cubic trajectory of Task 2 (T = 3 s). Plot τ_1 and τ_2 against time and report the peak torque of each joint.